

Bachelor Thesis

PATH PLANNING IN ROS 2 FOR INDUSTRIAL ROBOTICS

accomplished at



Bachelor's Degree Program
Automation Technology

by

Marcel Borkowski

52211664

supervised and reviewed by
FH-Prof. DI Dr. techn. Christian Gasser

Graz, June 2025

Marcel Borkowski

Signature

STATUTORY DECLARATION

I declare that I have authored this thesis independently, that I have not used other than the declared sources/resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources. While the core content, research, and conceptualization are entirely my own, AI language models were utilized for grammar, style, and clarity refinement after my initial drafting and deep revision. The AI model used for refinement was "Gemini 2.5 Pro Preview".

Marcel Bookowski

Signature

ABSTRACT

Modern mass production and high levels of consumer demand have made factory automation essential. Producing goods and parts on such a large scale simply wouldn't work without automated factories that handle everything from arranging machines on the factory floor to managing logistics of the flow of materials. In this dynamic landscape, the flexibility to adjust to continuously evolving production environments and frequently changing product specifications due to a wide range of offerings or the need to customize items for specific customer requirements, becomes crucial. Furthermore, the emerging trend of human-robot collaboration, particularly with collaborative robots (cobots), forces the development of production cells that can intelligently adapt to their surroundings. This includes responding to the changing positions of workpieces or tools and, critically, ensuring human and robot safety through the selection of appropriate, collision-free trajectories. Consequently, optimizing even seemingly minor aspects of the automation process can improve significantly overall benefits.

This thesis focuses on a specific side of this challenge: the development of a software solution demonstrating the capabilities of ROS 2, in combination with OpenCV for perception and MoveIt 2 for motion planning, to achieve dynamic path finding for an industrial robot. The core objective is to create software that automatically determines optimal, efficient, and safe trajectories for a robot to move between a defined start point (A) and a target end point (B). A key feature of this system is that the target point B is not fixed, and its position is dynamically localized in real-time using image processing techniques, allowing the robot to adapt to its changing workspace.

TABLE OF CONTENTS

ABSTRACT	ii
Table of Contents	i
1 Introduction.....	1
1.1 Research Objectives.....	2
2 ROS 2 in industrial Robotics	3
2.1 OpenCV	3
2.2 RVIZ.....	3
2.3 Gazebo	4
2.4 Universal Robot ROS 2 Driver.....	5
2.5 MoveIt	6
3 Path Planning	9
3.0 State-of-the-art in Path Planning	9
3.1 Path Panning	10
3.2 OMPLs Path Planners	11
3.3 Tree-based Planners	12
3.4 Motion Planning vs Path Planning	15
3.5 Cost Analysis for a Trajectory Planning.....	16
3.6 MoveIt Task Constructor.....	18
4 Image Processing.....	20
4.1 Storage Detection	21
4.2 Storage Localization	22
5 Example application	25
5.1 Communication Methods in ROS 2	25
5.2 Communication Architecture and Operational Flow	27
6 Conclusions and future work.....	30
Bibliography	31

1 INTRODUCTION

When setting up a new robot on a production line, there are generally two ways to program its movements. The first is called online programming. It's one of the older methods where a programmer physically guides the actual robot through the steps it needs to perform. They move the robot arm to a desired spot, record that position, and repeat this until the whole task path is mapped out. Once programmed, the robot repeats this exact path over and over very accurately. This method is fairly simple and doesn't demand deep programming knowledge. However, it has downsides: it's slow because you have to stop production, the programmer needs to be right there with the robot, and there's no guarantee the path chosen is actually the most efficient one ¹.

Another method is offline programming (OLP). This was developed to avoid stopping production and keep programmers out of the potentially dangerous robot work area. The idea is to create a 3D computer model (a digital twin) that's an exact copy of the real robot and its workspace. The programmer writes and tests the robot's instructions in this virtual simulation first. This speeds things up compared to online programming because the real robot keeps working. But OLP has its own issues. Real robots aren't perfect, so they might not move exactly like the simulation predicts. More importantly for path planning, a human still has to manually add points or guide the path in the simulation to make sure the robot doesn't crash into things. This means both online and offline programming work best for fixed production lines where the parts being made and the robot's surroundings don't change often. They meet the needs of mass production where the same task is done thousands of times, but they struggle if things need to change, as any unexpected object could cause a collision or target point would change. This was common before the current trend of Industry 4.0, back when factories first started using computers for basic automation (Industry 3.0) but still needed humans for overall control ².

Today, the Industry 4.0 trend emphasizes flexibility and faster setup times. This creates a need for a newer, third way of programming robots. This method aims to be more flexible and cut down on programming time even further. The idea is to use a virtual environment (the digital twin) that's directly connected to the real robot. By simply telling the software what the workpiece is and where it needs to start and end, the software should automatically figure out the best path. "Best" can mean fastest, most energy-efficient, or smoothest, while always making sure the robot won't collide with anything. Some advanced systems might even use sensors to adjust the path in real-time if the environment changes unexpectedly, called dynamic path planning.

This automatic path planning is the main topic of this bachelor thesis. It describes the idea and backend of how the software works. Using a digital copy of the robot (a UR5 model) and its work area, the goal is to create a tool that finds the most efficient, collision-free paths for the robot. This thesis, firstly, briefly covers

¹ (What is robot programming, 2022)

² (Shaw, 2024)

ROS 2 for robotics, controlling a UR5 robot, and using simulation. Then, it explains path planning: what it is, how paths are optimized, common algorithms used, and current difficulties with path planning. After that, the OpenCV plugin tool, for dynamic target finding, using image computing is presented. Finally, describes the results of running the developed software, and it's the possibility of connecting to a real robot. This thesis builds on the work from the first part of this bachelor project, titled: "*How ROS2 supports cross-platform compatibility in industrial robotics applications*", which investigates ROS 2 usage for a robotic system.

1.1 Research Objectives

This bachelor thesis details the development, implementation, and evaluation of an integrated ROS 2 based system designed for automated, collision-free path planning adapted for a Universal Robot 5 (UR5) manipulator. A core component of this attempt is the utilization of the MoveIt framework as the primary platform for motion planning, which involves setting up a complete digital twin of the UR5, establishing a realistic simulation environment using Gazebo, and employing RViz for 3D visualization and user interaction.

Furthermore, to enhance the system's adaptability to its operational environment, this work aims to incorporate fundamental image processing capabilities using the OpenCV library. This will involve developing methods for basic object detection and localization from camera input, allowing the planning scene to be updated to reflect changes in the workspace, thereby enabling more dynamic and responsive path planning.

A final key objective is to test the solutions generated by the path planning system. This validation will be conducted through the commissioning process entirely within the Gazebo simulated environment. Planned trajectories will be executed in this virtual setting to ensure their safety, correctness, and adherence to task requirements. The cheapest solution, based on the operator's constraints, will serve as the primary demonstration of the system's capabilities. The overarching goal remains to produce a robust software solution that automates the complex task of generating safe and efficient robot paths within a simulation, showing its distinguished capabilities in industrial robotics. Thereby reducing reliance on manual programming efforts and improving the potential for path optimization.

2 ROS 2 IN INDUSTRIAL ROBOTICS

Robot Operating System 2 (ROS 2) is a comprehensive software framework engineered for the development of advanced robotic applications. It comprises a set of software libraries and tools designed to facilitate state-of-the-art robotics functionalities. The open-source nature of ROS 2 affords a high degree of flexibility and freedom in software development, which is a primary motivation for its adoption as an integrating framework for various plugins and external software components essential for complex robotic systems. Within the context of this project, ROS 2 provides the foundational infrastructure to interconnect elements such as the digital twin of the robot, the simulation of its physical environment, the path planning engine, image computing, and the communication interface with the physical robot hardware ³.

As this thesis represents the continuation of a prior bachelor's project, a detailed description of ROS 2 is not provided here. To understand why ROS 2 was chosen for industrial robotics, it's recommended to read the first thesis part, *"How ROS 2 supports cross-platform compatibility in industrial robotics applications"* ⁴. For more detailed information about ROS 2 in general, the survey *"ROS 2 Key Challenges and Advances: A Survey of ROS 2 Research, Libraries, and Applications"* is a good resource.

2.1 OpenCV

One of the objectives of this project is to make the path finding fully automated and responsive to changing environment. To achieve this adaptability, a sensor capable of gathering real-time environmental data is required. Among the various sensors available, a standard camera was selected for this purpose. Cameras offer advantages such as ease of mounting, the ability to monitor a significant portion of the robot's workspace, and considerable flexibility in terms of software-based image processing.

For image processing tasks, the widely recognized and powerful OpenCV (Open-Source Computer Vision Library) is utilized. OpenCV is one of the most extensive open-source libraries dedicated to computer vision applications. Its origins are closely linked with the robotics community, having been developed with significant contributions from the same organization that initiated ROS, and it has been a fundamental in ROS-based systems since ROS 1 ⁵.

2.2 RVIZ

Another goal of this project is to create software that can find a collision-free path between two points (a start and a goal) within a known environment, represented as a digital twin. To make this process observable and interactive for a human user, a 3D visualizer is essential. For this purpose, RViz is utilized. RViz is a 3D visualization tool designed for robots using ROS. It works by subscribing to ROS messages

³ (Robotics O. , ROS - Robot Operating System, 2021)

⁴ (Al-Batati, Koubaa i Abdelkader, 2024)

⁵ (OpenCV-team, 2000)

published by various system components and then displaying the corresponding 3D information, such as robot models, sensor data, and planned paths, in its visualization window ⁶. Effectively, RViz provides a human-machine interface that allows users to monitor and interact with the digital twin of the robot and its environment.

Figure 1 presents an example of the RViz interface. It is highly customizable, and the described layout is just one of many possible configurations for its Graphical User Interface (GUI). RViz includes a wide range of built-in display types and tools. For instance, the example in Figure 1 shows an image display panel (visible in the bottom-left corner), which could render a live stream from a camera. Additionally, a common panel (on the top-left) displays a tree-like module listing all active tools and allowing users to configure their respective settings and properties.

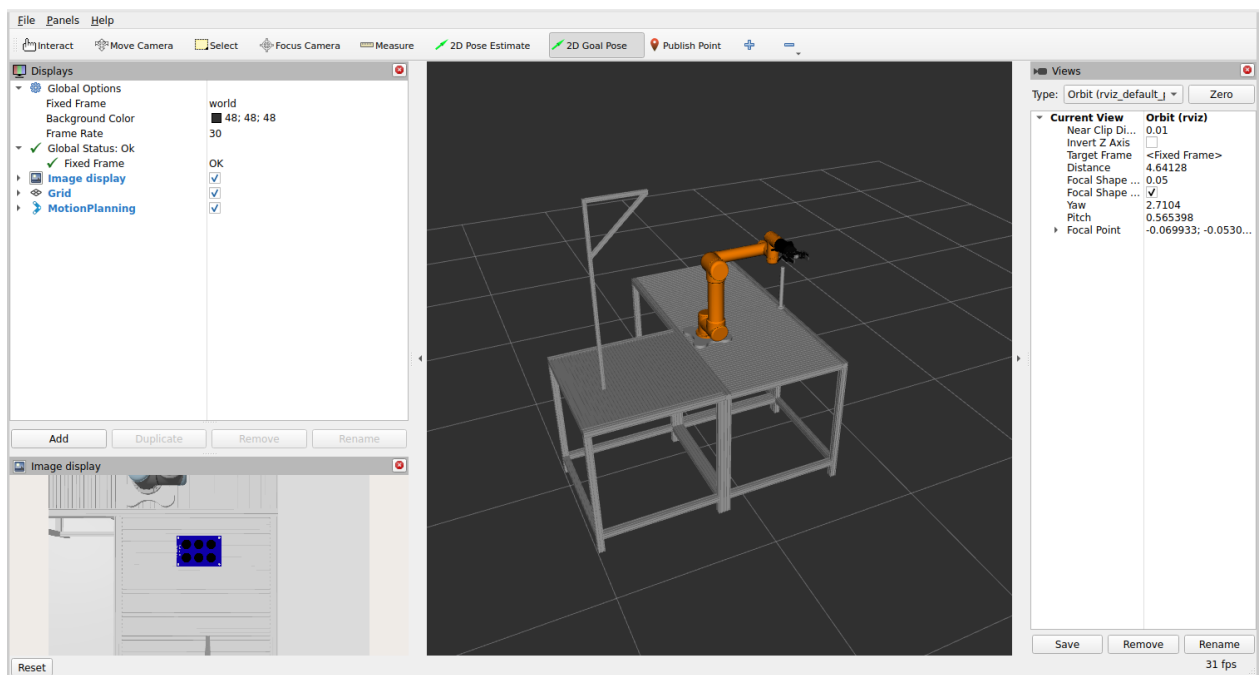


Figure 1: RViz visualization of the UR5 robot and its workspace as a digital twin. Source: Own image.

2.3 Gazebo

To simulate the physical world and the robot's interaction with it, the Gazebo simulator is employed. Gazebo is a valuable tool during software development and for verifying the output of planning algorithms before deploying them to a physical robot. As a simulation software, it aims to replicate real-world physics and behaviors, including gravity, object inertia, friction forces, and potentially other environmental factors. This allows for testing control strategies and path plans in a safe, virtual environment, enabling thorough evaluation and continuous integration tests without the risk of damaging physical hardware ⁷.

⁶ (Hershberger, Gossow i Faust, 2019)

⁷ (Robotics, 2002)

Figure 2 showcases the customized GUI of the Gazebo simulation environment, designed to the specific needs of this project. The main view presents a real-time 3D simulation of the robot's workspace. On the right side of the interface, an entity tree panel is visible, listing all the objects and models currently present within the simulated environment. When an entity is selected from this tree, its basic properties, such as its name, pose (position and orientation), and other relevant data, are typically displayed in a dedicated properties panel, often located in the upper-right portion of the GUI.

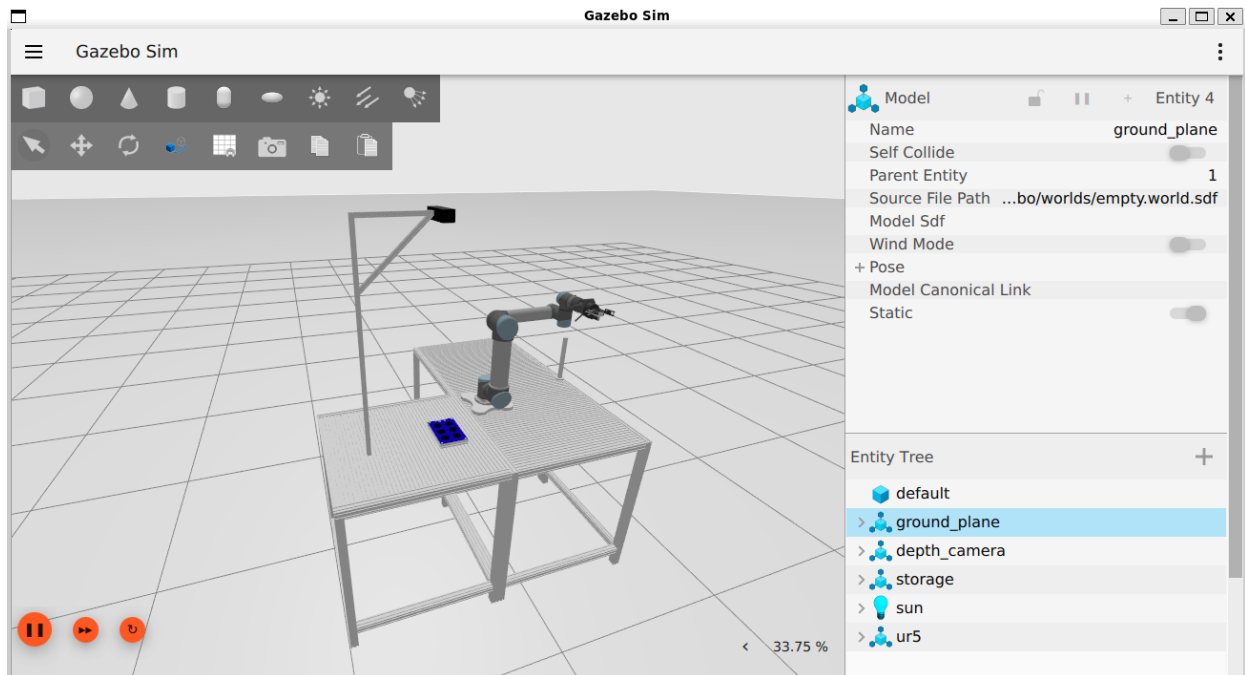


Figure 2: Gazebo (real-world) simulation of the UR5 robot and its workspace as a digital twin. Source: Own image.

2.4 Universal Robot ROS 2 Driver

Controlling a Universal Robot (UR) arm externally using ROS 2 requires two main components. The first is the Universal Robots ROS 2 Driver, which is installed on the control PC. This driver enables the monitoring and control of UR robots from external software by providing an interface to read the robot's joint states (from the physical twin) and transmit this data to its digital representation for computation and planning. The UR driver is built upon standard C++ libraries. ROS 2 supports multiple programming languages, including C++ and Python, this facilitates straightforward integration. This driver allows communication between different ROS 2 nodes, enabling direct interaction with the digital twin and path planner for status monitoring and task execution ⁸.

The second component is URCap External Control. This URCap is a software plugin installed directly onto the UR robot's control system (teach pendant). It acts as the on-robot interface that facilitates communication with the external ROS 2 driver and the underlying Robot Client Library used by the driver.

⁸ (A/S, 2024)

This URCap supports UR CB3 series robots and the newer e-Series robots, effectively enabling the physical robot to be controlled by the ROS 2 system running on the PC ⁹.

2.5 MoveIt

The final key software component utilized is MoveIt 2. MoveIt 2 is a comprehensive robotic manipulation platform for ROS 2, providing a suite of functionalities including motion planning, manipulation capabilities, 3D perception integration, kinematics solvers, and interfaces for control and navigation. A core feature of MoveIt is its plugin-based architecture for motion planners, which allows users to readily connect and utilize different planning algorithms from various libraries (see Figure 3). In this project, MoveIt serves as the central engine that processes environmental data and task requirements to compute desired trajectories for the robot, primarily through ROS 2 actions and services ¹⁰.

To initiate planning with MoveIt, a motion plan request is typically utilized. This request specifies either a target joint space configuration or a desired Cartesian pose for the robot's end-effector. Upon receiving this request, MoveIt invokes the configured motion planner (for example an algorithm from OMPL) to generate a collision-free path, considering potential collisions with the environment, self-collisions, and any attached objects (such as a gripped workpiece). The planner operates within the constraints defined by the user and the robot's model. It is important to note that the output from the MoveIt planning pipeline is typically a trajectory, not purely a geometric path. This means it includes timing information, specifying desired velocities and accelerations at the joint level ¹¹. This time parameterization is often achieved through post-processing steps like Time-Optimal Trajectory Generation, effectively bridging the gap between a purely geometric path and a dynamically executable robot motion ¹². MoveIt utilizes these techniques to produce smooth trajectories with continuous velocity profiles, which helps to avoid jerky movements or velocity spikes that can occur when paths are manually defined by humans ¹³.

Figure 3 illustrates the RViz interface integrated with the MoveIt 2 Motion Planning display plugin. On the left side of the interface, a dedicated panel is visible, which allows for the selection and configuration of various motion planning parameters. This includes options for choosing the planning library and specific algorithm. In the described example, OMPL is selected as the planning library (OMPL and its algorithms are discussed in detail in the next Path Planning chapter). Within the main 3D view, which displays the digital twin of the UR5 robot, an orange "ghost" representation of the robot arm indicates the desired goal state or target pose. This goal state can often be interactively manipulated by the user, for instance, by dragging the light green interactive marker sphere located at the end-effector of the ghost model.

⁹ (fmauch, 2021)

¹⁰ (Robotics P. , 2025)

¹¹ (Robotics P. , Motion Planning, 2025)

¹² (Robotics P. , Time Parameterization, 2025)

¹³ (Owen-Hill, 2015)

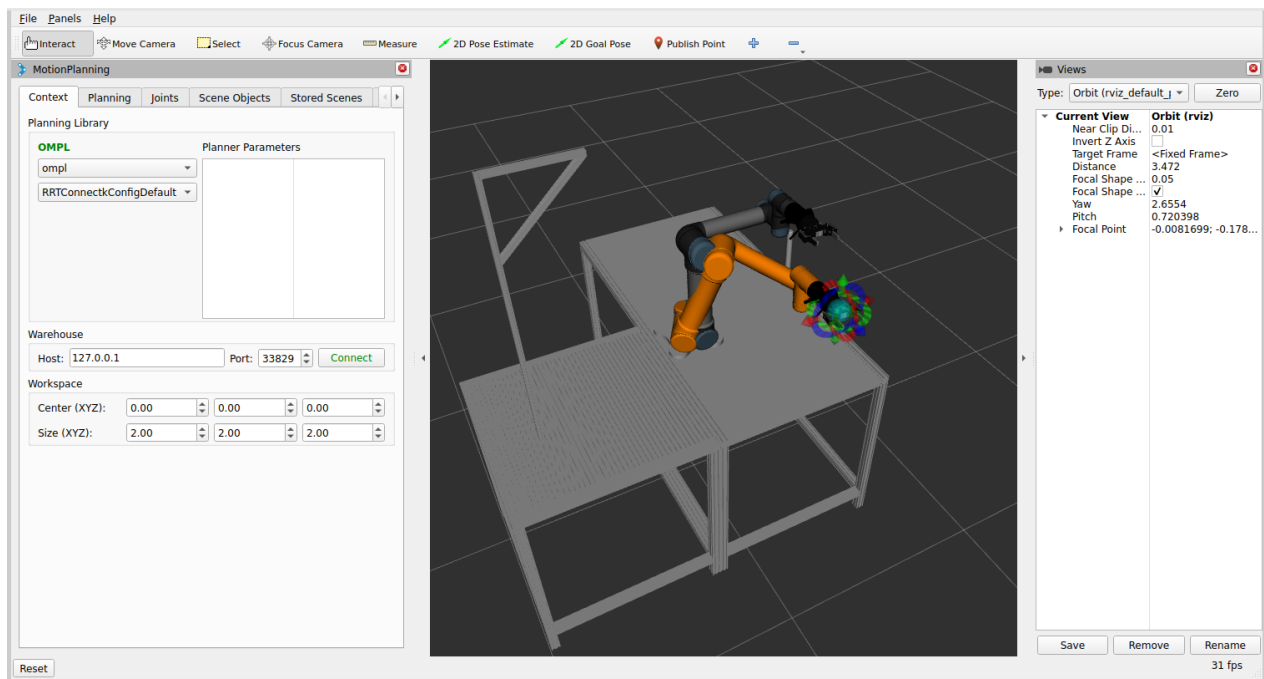


Figure 3: The RViz visualization integrated with the MoveIt Motion Planning plugin. Source: Own image.

A significant advantage of these software components (RViz, Gazebo, UR Driver, OpenCV, and MoveIt) is their seamless integration with ROS 2, establishing them as standard tools within the robotics development community ¹⁴. This compatibility simplifies their incorporation into the overall software architecture. Communication between these components primarily relies on ROS 2's publish/subscribe and service/action client-server models, which facilitate data exchange between nodes in the ROS graph.

In the developed software, the Gazebo simulation environment can be used interchangeably with the physical robot setup. While there are implementation differences for interacting with simulation versus real hardware, the overall software infrastructure and control flow remain largely consistent. The process begins with a perception phase. In this phase, information about the robot's current state (joint motor positions) is acquired, and critically, visual data from a camera is processed using OpenCV to understand the robot's surroundings. This provides an understanding of the workspace and identifies navigable free space. Next, the task definition phase involves specifying the start and goal configurations or poses for the robot's end-effector. This might involve the data gathered from image processing and object detection, such as moving part storage. Which detail coordinates, are derived from live camera commissioning. That can define the goal position, while the start position is well defined in the fixed frame. With these inputs, the MoveIt plugin, utilizing the selected path planning algorithm, computes a potential collision-free trajectory. Once a valid path is found and parameterized into a trajectory, it is translated into a sequence of commands understandable by the robot controller. This command sequence can then be sent to the Gazebo simulation for commissioning, by verifying that the execution is safe and meets user expectations. If the simulated execution is satisfactory, the same set of instructions can be dispatched to the physical robot via the

¹⁴ (Robotics, ROS 2 Documentation: Jazzy, 2025)

Universal Robots ROS 2 Driver. If the trajectory requires improvement (if it's too slow, jerky, or encounters issues in simulation), the planning and refinement process can be iterated until a satisfactory solution is achieved. Figure 4 depicts a simplified architecture of this communication flow, illustrating the interaction between the simulation, planning, and control layers.

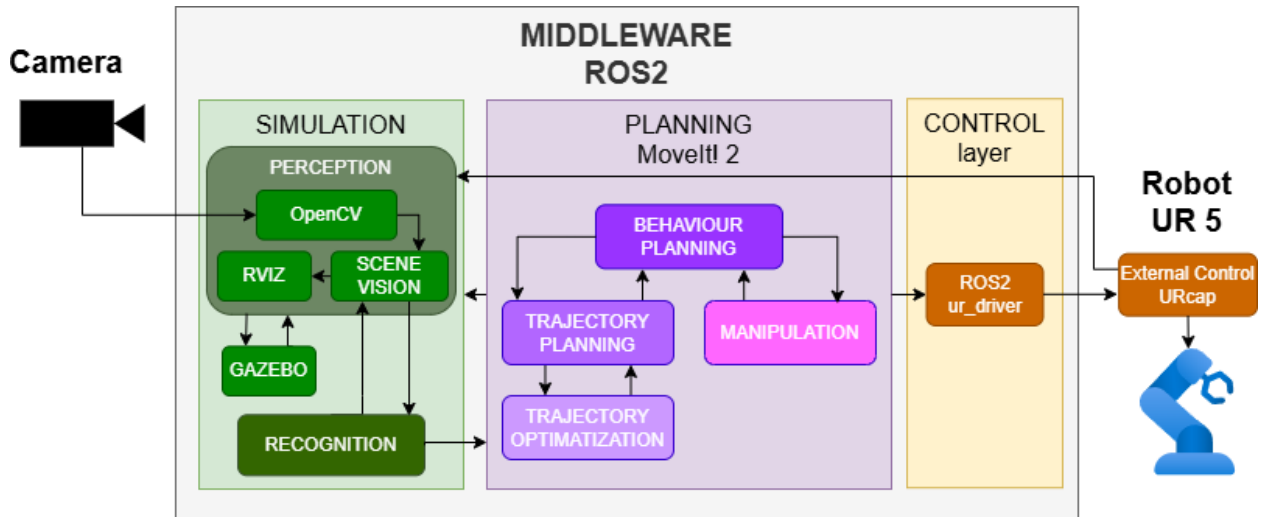


Figure 4: A simplified communication architecture for the developed software, illustrating the simulation, planning, and control layers for interaction with the physical robot or its simulated digital twin. Source: Own image.

3 PATH PLANNING

To provide a comprehensive understanding of the path planning challenges addressed in this project, the following preview section will first outline conventional approaches to robot programming and path definition. Afterwards, the limitations of these traditional methods, particularly concerning efficiency and adaptability, will be discussed. This will then lead to an explanation of the automated path planning solutions employed within this project as an effective alternative.

3.0 State-of-the-art in Path Planning

Offline programming (OLP) represents a predominant standard in industrial robotics, principally attributable to its ability to address major limitations of traditional online teach pendant programming. By shifting the programming process to a virtual simulation, utilizing a digital twin of the physical robotic cell, the OLP significantly reduces production downtime, as the real robot can continue operating while new tasks are developed ¹⁵. Furthermore, it enhances programmer safety by removing the need for direct physical presence within the potentially hazardous robot workspace during the initial programming stages.

However, despite these advantages over online methods, the conventional approach to path definition within OLP often falls short of producing truly optimal results. Traditionally, even within the OLP simulation, the programmer typically defines the robot's path by virtually specifying a sequence of key points or configurations, mirroring the point-by-point process of online teaching ¹⁶. While this manual approach is generally faster and less susceptible to errors than physical teaching due to the feedback and visualization capabilities of the simulation, it remains significantly dependent on the programmer's intuition and experience to define a workable path.

The critical limitation is that the programmer's primary focus during manual path definition is typically on achieving task completion and ensuring basic collision avoidance, rather than rigorously optimizing for criteria such as cycle time, energy consumption, or trajectory smoothness ¹⁷. It is inherently challenging for human programmers to intuitively identify the globally optimal path within the robot's high-dimensional configuration space, particularly when required to simultaneously satisfy joint limits, velocity constraints, and collision avoidance mandates while optimizing for performance metrics. Consequently, the manually defined path in OLP often represents only a feasible solution, not necessarily the most efficient one, thereby leaving significant scope for improvement through automated path planning algorithms designed to explicitly optimize for desired criteria.

¹⁵ (What is offline robot programming (OLP)? A complete guide, 2025)

¹⁶ (About offline programming, brak daty)

¹⁷ (Gîrbacia, Duguleană i Stavăr, 2011)

3.1 Path Panning

While the start and end configurations of a robotic task are typically defined by operational requirements, the trajectory connecting them can vary significantly depending on performance objectives and environmental constraints. There exist numerous potential paths between any two given points in a robot's workspace. Consider the intuitive task of moving between two points, perhaps analogous to opposite corners of a square. For simple travel, a straight line appears to be the most obvious and shortest connection. This holds true if considering only the Cartesian distance for the end-effector.

However, the complexities of multi-joint robotic arms introduce additional considerations that make path generation non-trivial. Achieving a linear, straight-line (LIN) motion in Cartesian space requires precise, often complex, coordination of multiple joints, potentially limiting the overall speed to ensure the end-effector follows the prescribed path. Conversely, a Point-to-Point (PTP) motion allows each joint to move independently towards its target angle, often concurrently and potentially at maximum individual joint speeds. While this typically results in a curved Cartesian path for the end-effector (see Figure 5), PTP motion can often achieve the desired end configuration faster and potentially with less energy consumption compared to a constrained linear move for the same start and end points, as joints are less restricted ¹⁸. It is crucial to understand, however, that PTP motion describes an execution method, not a path planning algorithm itself. While the controller generates intermediate joint values during a PTP move via interpolation, these are not explicitly calculated to avoid collisions along the resulting trajectory segment. Ensuring a collision-free path, especially in complex environments, typically requires a dedicated path planning algorithm to compute a sequence of intermediate waypoints, which might then be traversed using PTP or other motion types ¹⁹.

¹⁸ (Henry, 2025)

¹⁹ (MRobotics, 2025)

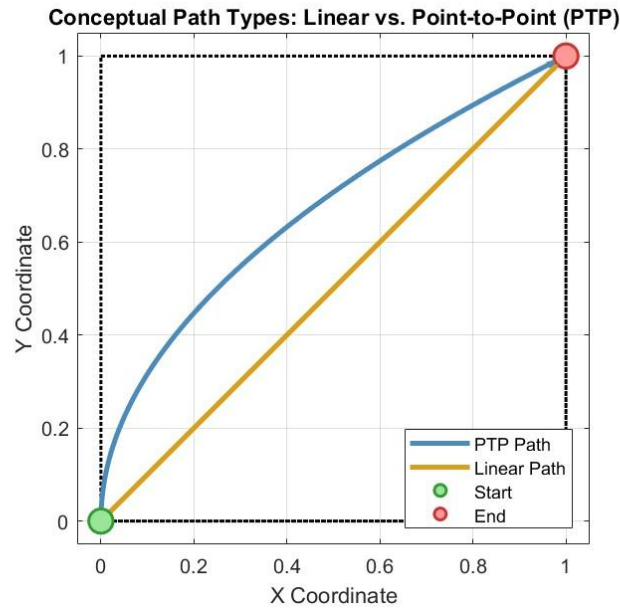


Figure 5: Visualization of Linear (LIN) vs Point-to-point (PTP) movement, in 2D cartesian representation. Source: Own image.

The definition of an "optimal" path becomes crucial and depends heavily on the application, incorporating factors such as minimizing execution time (cycle time), reducing energy consumption, ensuring path smoothness (to reduce wear or vibrations), or maintaining specific tool orientations. The problem escalates further with the introduction of obstacles within the robot's workspace. The planning algorithm must then find a route that not only connects the start and end configurations but also guarantees collision avoidance with all objects in the environment, respecting defined safety margins.

Therefore, path planning in robotics is formally defined as the computational problem of finding a continuous, collision-free sequence of configurations (a path) from a given start state to a desired goal state, often while simultaneously optimizing one or more performance metrics according to user-defined criteria. Achieving a single path that perfectly optimizes all possible criteria (shortest, fastest, smoothest, most energy-efficient) is generally impracticable due to inherent trade-offs between these objectives. Practical path planning therefore involves finding a suitable balance based on task priorities. Addressing this complex and long-standing challenge has led to the development of numerous distinct path planning algorithms, each employing different strategies and suited for specific types of problems and optimization goals.

3.2 OMPLs Path Planners

Movelt is designed to interface with several distinct planning libraries and algorithms ²⁰. The primary focus of the path planning investigation within this thesis lies on the Pilz Industrial Motion Planner and the Open Motion Planning Library (OMPL). This decision is motivated by their status as standard, well-integrated,

²⁰ (Planners Available in Movelt, brak daty)

and well-supported planning frameworks within the MoveIt and ROS 2 ecosystem. While the Pilz planner provides deterministic methods for common industrial trajectories (LINear and CIRCular), OMPL offers a comprehensive suite containing a wide variety of sampling-based algorithms.

The following terminology derived from “*Open Motion Planning Library: A Primer*” prepared by Kavraki Lab, at Rice University, might be useful for better understanding of the path planning topic (page: 4):

- a. **Workspace:** The physical space that the robot operates in. It is assumed that the boundary of the workspace represents an obstacle for the robot.
- b. **State space:** The parameter space for the robot. This space represents all possible configurations of the robot in the workspace. A single point in the state space is a state.
- c. **Free state space:** A subset of the state space in which each state corresponds to an obstacle free configuration of the robot embedded in the workspace.
- d. **Path:** A continuous mapping of states in the state space. A path is collision free if each element of the path is an element of the free state space.

The inclusion of OMPL is particularly relevant because its sampling-based algorithms are fundamental to addressing the complex challenge of finding collision-free paths in the high-dimensional configuration space characteristic of industrial robots, especially when navigating environments cluttered with obstacles.

Sampling-based planners operate by random sampling configurations within the robot's valid configuration space. These samples are then incrementally connected to form a graph or tree structure, provided the connecting path segments are collision-free and respect the robot's kinematic constraints, ultimately aiming to bridge the start and goal configurations specified in the planning query. A key characteristic of many sampling-based methods is probabilistic completeness. This property means that if a valid path between the start and goal configurations exists, the probability of the algorithm finding such a path approaches 1 (0 - no solution; 1 - all solutions found) as the number of samples generated increases infinitely. Consequently, there is a fundamental trade-off between the allocated computation time (which dictates the number of samples explored) and the chance of finding a solution or improving its quality, by reducing a path length. Finding an effective balance between computational effort and solution quality is therefore critical for deploying these planners effectively in practical applications ²¹.

3.3 Tree-based Planners

Given the wide variety of specific tree-based planning algorithms, this section outlines their general operational principle rather than detailing a single implementation. These methods typically initiate by establishing the robot's starting configuration as the root node of a search tree. The tree is then incrementally expanded using a guided strategy, often incorporating heuristics, to explore the collision-free configuration space and establish a connection to the goal configuration.

²¹ (Lab-Kavraki, 2025)

A purely random or 'blind' search of the vast configuration space would be computationally prohibitive. Therefore, to improve efficiency, the expansion process is often biased, increasing the probability of sampling or extending the tree in the general direction of the goal state. This iterative expansion typically involves repeatedly sampling a random configuration, identifying the nearest existing node in the tree to that sample, extending the tree from this nearest node towards the sample by a small step, and adding the resulting new configuration (and the connecting segment) to the tree if the segment is collision-free. The tree therefore grows outwards with branches exploring different regions of the configuration space, reflecting the algorithm's attempt to explore randomly yet rapidly, which gives rise to the common algorithm name Rapidly-exploring Random Tree (RRT). Figure 6 illustrates a conceptual example of such a randomly grown tree rooted at a starting node within a 2D workspace²². Where a gray objects are a dead spaces, impossible to be reached by the robot, for example obstacles and graph with orange and light blue nodes representing a growing tree – path finder.

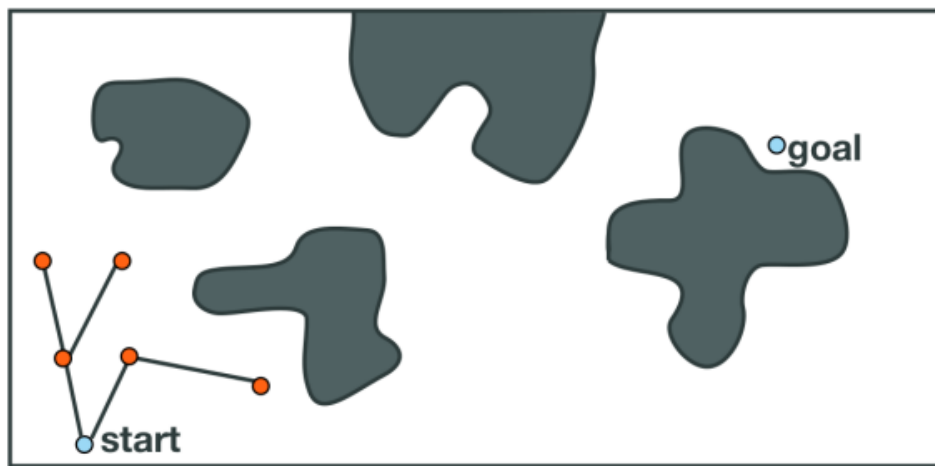


Figure 6: "The sampling observed after the first few samples have been connected. Random samples are connected to the tree using an expansion heuristic." Source: "*Open Motion Planning Library: A Primer*," page 7, prepared by Kavraki Lab, at Rice University.

Once an initial connection between the start and goal configurations is found by the planner, this first feasible path is often suboptimal. Subsequently, an optimization process, either as a distinct post-processing step or inherently integrated within certain algorithms, seeks to improve the path quality based on specified criteria. For example, optimization might aim to identify the trajectory with the shortest geometric length among possible variations or refinements (see Figure 7).

²² (Lab-Kavraki, 2025)

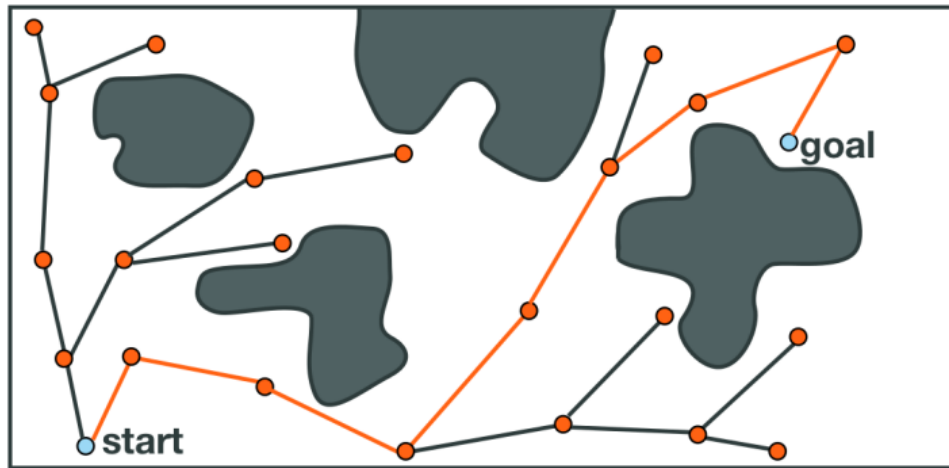


Figure 7: "Scenario where the goal is selected during the sampling process, and is connected to the tree, ending the search."

Source: "Open Motion Planning Library: A Primer." page 8, prepared by Kavraki Lab, at Rice University.

To improve the efficiency of the basic RRT algorithm, the RRT-Connect variant was developed. This method employs a bidirectional search strategy, simultaneously growing two trees: one rooted at the start configuration T_s and the other at the goal configuration T_g (see Figure 8). A key element differentiating RRT-Connect is its aggressive 'Connect' heuristic, which complements the standard RRT expansion.

After one tree successfully extends to a new node by reaching towards a random sample, the Connect heuristic attempts to extend the other tree directly towards this newly added node, often over multiple steps, if necessary, in a greedy effort to quickly establish a connection between the two trees. Both trees are maintained and expanded, typically alternating between them, until they successfully connect, thereby finding a complete path solution. This bidirectional exploration of the free configuration space, combined with the strong bias introduced by the greedy connection attempts towards the opposing tree, significantly accelerates the planning process compared to the unidirectional growth of the standard RRT algorithm ²³.

²³ (Kuffner i LaValle, 2000)

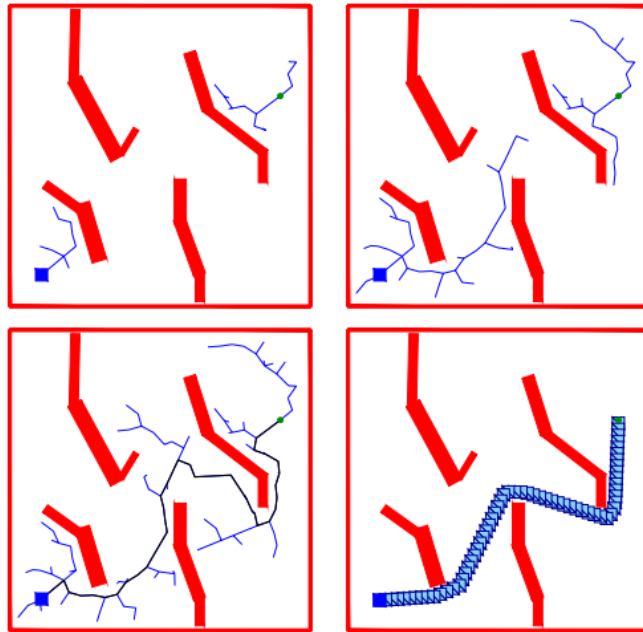


Figure 8: "Growing two trees towards each other." Source: "*RRT-Connect: An Efficient Approach to Single-Query Path Planning.*" page 3, prepared by Kuffner and LaValle for IEEE conference in USA.

RRTConnect is fully implemented within the OMPL planning library, which serves as the default planning framework for MoveIt. Due to this robust integration and its proven effectiveness in rapidly finding feasible solutions, Rapidly-exploring Random Tree Connect (RRTConnect) was selected as the primary path planning algorithm utilized in the project and in the next chapters ^{24 25}.

For clarity of explanation and enhanced visualization, the fundamental concepts of the path planning algorithms discussed herein are primarily illustrated using 2-dimensional (2D) examples. It is important to note, however, that the underlying theoretical principles extend directly to higher-dimensional spaces, such as the 3-dimensional (3D) environments utilized by the motion planner within this project. The transition to 3D involves an additional spatial dimension, leading to more complex geometric calculations, but the core algorithmic logic remains consistent.

3.4 Motion Planning vs Path Planning

The terms motion planning and path planning are often mistaken and used interchangeably, with many assuming they mean the same thing. While both terms relate to how a robot gets from point A to point B, they generally describe two separate processes in robotics.

Path planning, according to the previously mentioned definition, is the process of constructing a geometric path from a starting point to an ending point, often by computing intermediate points. It answers the

²⁴ (OMPL, 2022)

²⁵ (Planners Available in MoveIt, brak daty)

questions of *"what sequence of configurations avoids obstacles?"* and *"how can the destination be reached without collision?"*.

Motion planning, on the other hand, typically refers to the process of determining the time-based sequence of states or control actions needed to actually follow the planned path. This involves finding solutions to questions such as, *"how should the motors move over time to follow the given path?"* or *"what velocities and accelerations are required while respecting the robot's limits?"*. It's important to note that these processes are interdependent. Simply put, the path planner outputs the route to follow to avoid obstacles, while the motion planner determines how to execute that route in terms of timed movements and motor control ²⁶. The mirror name of path in motion planning is trajectory.

3.5 Cost Analysis for a Trajectory Planning

Standard RRT algorithms, while effective in finding collision-free paths, are not inherently designed for optimality. Their primary goal is to establish a feasible, obstacle-avoiding route. If it is desired to find the optimal trajectory in terms of execution time, energy consumption or the minimal distance follow by end effector, additional mechanisms for path evaluation and selection are required ²⁷. The final selected path, and its corresponding trajectory, should represent the best choice from a set of potential candidate solutions, all of which would successfully and safely guide the robot from point A to point B. To determine the "best" solution based on specific, predefined criteria, an algorithm or method must be applied to evaluate and rank these candidates.

Figure 9 illustrates a MoveIt architecture for trajectory optimization, involving cost calculation and the selection of an optimal path. The diagram shows how, starting from a motion plan request module (top-left) that uses planning scene information. It is then populated as a problem info module, which covers various terminal information, which refers to further cost and constrain analysis (such as joints velocity or joint limits, etc.). Which next are converted into specific functions, such as those derived from joint position equality const, seen in add objective terms. That ends up as defined problems with cost functions and constraints, in the construct problem module, ultimately leading to a ready-to-execute trajectory for the robot ²⁸.

²⁶ (Marcelina, 2022)

²⁷ (Dellaert & Hutchinson, 2023)

²⁸ (Said, Talj, Francis, & Shraim, 2022)

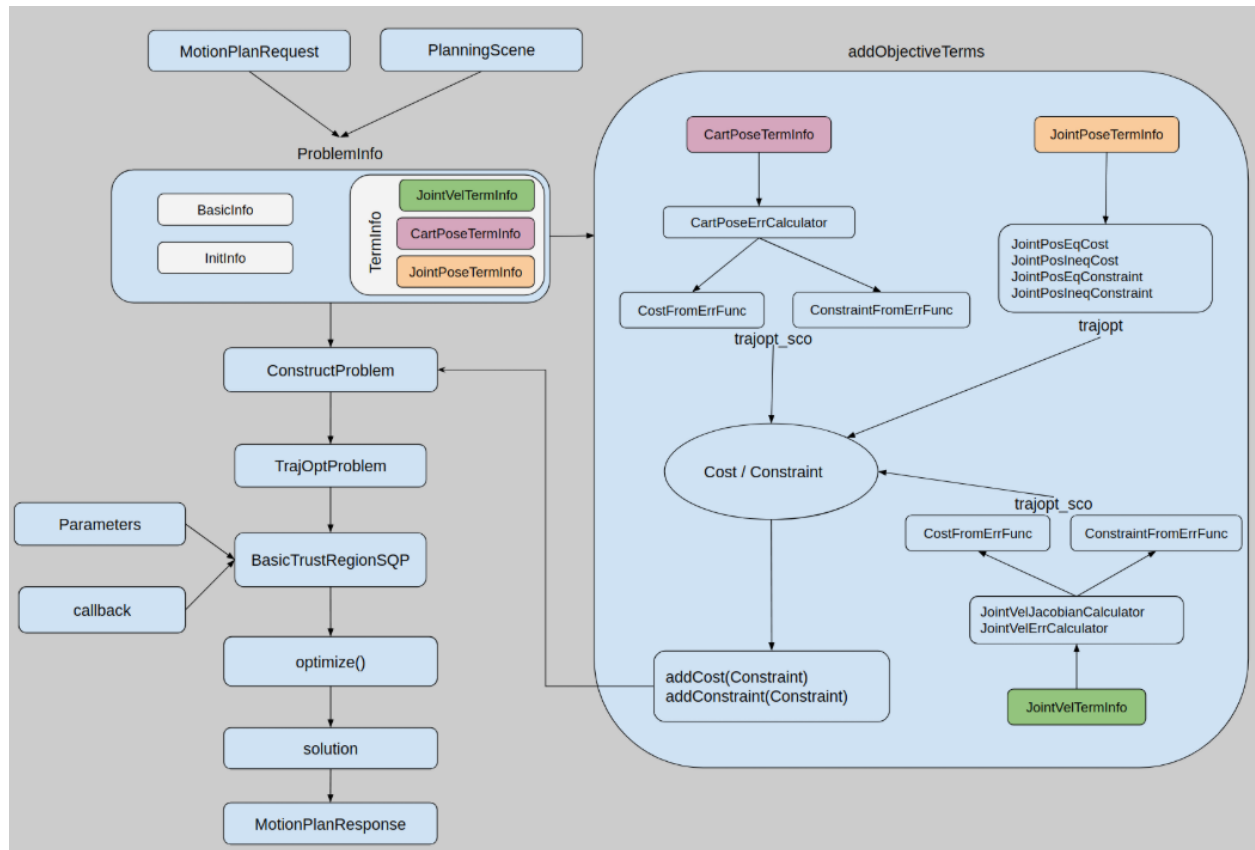


Figure 9: "The following flowchart illustrates how the problem of trajectory finding gets constructed". Source: MoveIt documentation, from Trajectory Optimization Planner documentation.

In MoveIt, the cost analysis is included in trajectory optimization. It is defined by a set of two functions, cost (COST) and constraints (CNT). The first function is the objectives that are supposed to be minimized for better performance, when CNT are the ones that must be followed and satisfied by the outcome solution. Constraints typically include elements such as joint position limits, maximum joint and end-effector velocities/accelerations, defined start and goal state requirements, and maintaining a minimum safe distance from obstacles.

The cost function, on the other hand, is often a weighted sum of various factors that are desirable to minimize for a better-quality trajectory. Common costs considered during this analysis include minimizing joint acceleration and velocity changes, which directly impact energy consumption, path length, trajectory duration, clearance (cost is inverse of distance to collision) and many more ²⁹. Jerky movements, characterized by frequent, rapid accelerations and decelerations, are generally undesirable due to inefficient energy use. Furthermore, collision avoidance, keeping a bigger distance from obstacles might be considered as a better solution, with a lower (better) cost score ³⁰.

²⁹ (Robotics P. , MoveIt Task Constructor, 2024)

³⁰ (Sphinx, 2017)

3.6 MoveIt Task Constructor

The last piece of the puzzle in the project of finding trajectories is MoveIt Task Constructor (MTC). It is a sophisticated framework within MoveIt designed to simplify the planning of complex robotic tasks by decomposing them into a sequence of smaller, interdependent subtasks or stages. This approach allows for a structured definition and solution of complex manipulation problems. The information in this subchapter is primarily drawn from the official MoveIt Task Constructor documentation, "MoveIt Task Constructor", provided by PickNik Robotics ³¹.

A complete task, such as picking up an object and placing it elsewhere, can be broken down into individual steps like "approach the object," "close the gripper," "lift the object," "move to the target position," "release the object," and "return to a home position." MTC conceptualizes such a task as a sequence of stages, where each stage represents a single computational step or a specific action the robot needs to perform (e.g., "open gripper"). Each stage aims to compute a sub-solution or a set of candidate solutions for that particular step ³².

MTC utilizes three primary types of stages to handle the diverse aspects of task-level planning, enabling the creation of complex robot behaviors through their interaction:

1. **Generator Stages:** These stages do not receive inputs from adjacent stages. Instead, they independently compute solutions (robot states or poses) and can pass these solutions both forward and backward along the task sequence. Examples of generator stages include:
 - a. **Current State:** Captures the robot's current state to serve as a starting point for subsequent planning within the task.
 - b. **Generate Pose:** Computes various possible poses for the robot, such as different grasp poses for an object or possible approach poses for a specific subtask.
2. **Propagating Stages:** These stages receive candidate solutions from a neighboring states, perform a computation or plan a motion based on that input, and then propagate the resulting new solutions to the neighbor on the opposite side. Depending on their implementation, some propagating stages can pass the solution forward, backward, or in both directions. An example is:
 - a. **Move Relative:** Takes an input robot state (the end-effector pose) and plans a relative motion (a short Cartesian move of a few centimeters in a specified direction, often used for precise approach or retreat motions).
3. **Connecting Stages:** These stages do not generate new solutions or propagate them in the same way as the other types. Instead, their primary function is to attempt to find a collision-free motion plan that connects an input start state (from one adjacent stage) to an input goal state (from another adjacent stage). They are frequently used to bridge gaps between different phases of a task, such as moving from a current pose to a new goal position.

³¹ (Robotics P. , MoveIt Task Constructor, 2024)

³² (automaticaddison, 2024)

MTC also allows for the hierarchical organization of these stages using containers. Containers define the order and logic of stage execution. Currently, MTC supports two main types of containers:

- Serial Container:** Organizes stages in a strictly linear sequence, where solutions must successfully pass from one stage to the next to form a complete task solution.
- Parallel Container:** Allows for the definition of multiple alternative sub-sequences of stages. This enables the framework to explore different strategies (in parallel) or alternative solutions for the same segment of a task.

Furthermore, MTC incorporates mechanisms for cost calculation, allowing solutions generated by stages to be evaluated and ranked. The three stage types described are fundamental building blocks. By combining these primitives within containers, users can construct detailed task plans, such as the "pick and place" example illustrated in Figure 10, where different arrows and bars represent various stage types and their connections ³³.

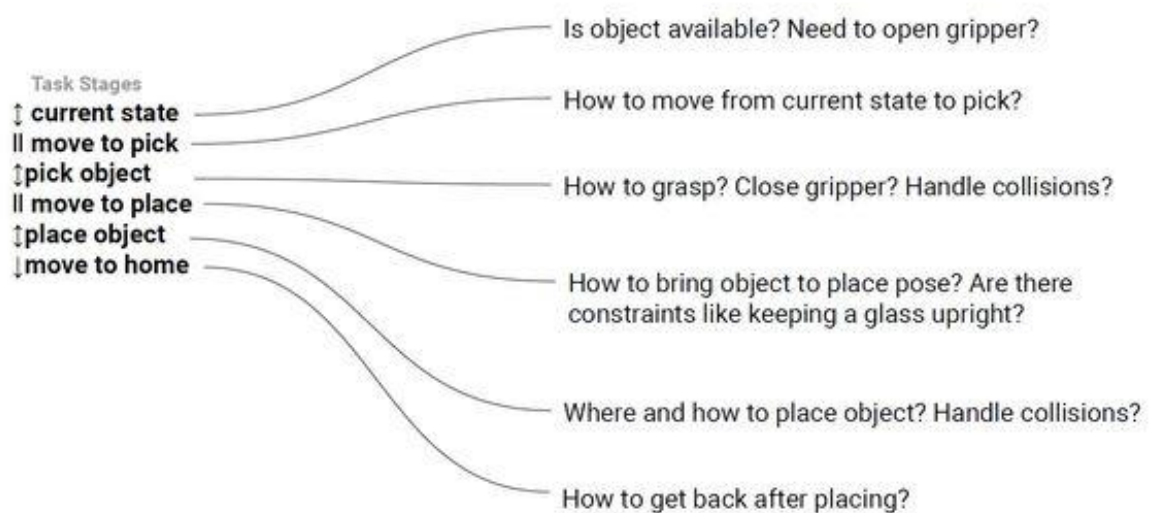


Figure 10: Example of how stages might be organized for a simple pick and place task. Source: PickNik AI (<https://picknik.ai/>)

³³ (automaticaddison, 2024)

4 IMAGE PROCESSING

A camera is employed in this project to detect the dynamically changing position of a specific object within the robot's workspace. "Detecting" in this context refers to finding the object in the image, whereas localization means determining the object's 3D coordinates relative to the world coordinate system. The target object for detection, henceforth referred to as the "storage unit," has well-defined physical characteristics: it is a rectangular cuboid with dimensions 220mm x 150mm x 27mm. Its top surface is blue and features six black cylindrical indentations (fossettes), each with a 46mm diameter and 3mm depth (see Figure 11). This storage unit is designed to temporarily hold workpieces within these indentations. These known geometric and visual features provide the necessary constraints for the detection algorithm.

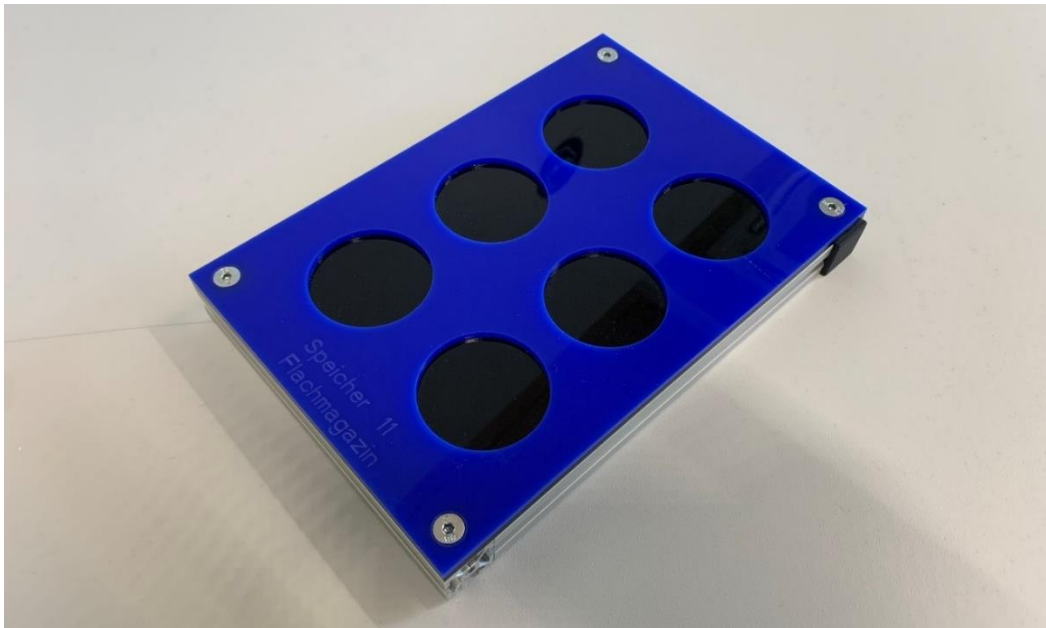


Figure 11: The movable storage unit. Source: Own image.

The experimental setup involves an RGB camera positioned above the workspace, providing a clear, top-down view of the storage unit and a significant portion of the robot's operational area. For optimal detection performance, it is beneficial to maintain a surrounding environment with a relatively uniform, flat color that contrasts with the storage unit and any workpieces. This contrast improves the accuracy of the object detection process (see Figure 12 for a conceptual illustration of the setup).

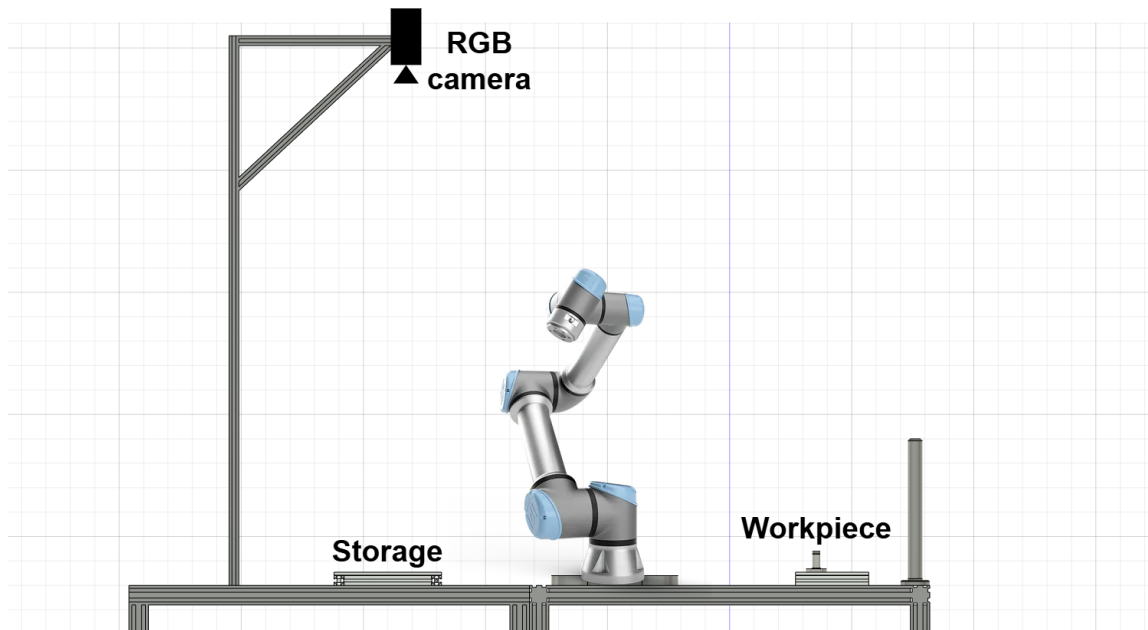


Figure 12: Conceptual illustration of the experimental setup, including the UR5 (not scaled). Source: Own image.

4.1 Storage Detection

Described setup enables real-time determination of the storage unit's position. The primary approach for detecting the storage unit begins with color-based image segmentation. This involves applying a color mask to the incoming camera image to isolate pixels corresponding to the known blue color of the storage unit, thereby preparing the data for subsequent processing steps.

A common technique for this is color thresholding in the HSV (Hue, Saturation, Value) color space. The HSV model represents colors in a cylindrical coordinate system, which is often more robust to lighting variations for color segmentation tasks compared to the standard RGB model. By defining specific lower and upper boundary values for Hue, Saturation, and Value that correspond to the target blue color, a binary mask is created. Applying this mask to the image effectively isolates regions matching the desired color profile, significantly increasing the contrast between the blue storage unit and its surroundings (see Figure 13). A potential challenge arises if other objects with similar blue hues are present in the camera's field of view, such as the light blue components of the UR5 robot. To mitigate this, the HSV boundaries must be carefully calibrated to be selective for the specific dark blue shade of the storage unit, effectively filtering out other blue elements ^{34 35}.

³⁴ (HSL and HSV, 2025)

³⁵ (Veeramachaneni, 2024)

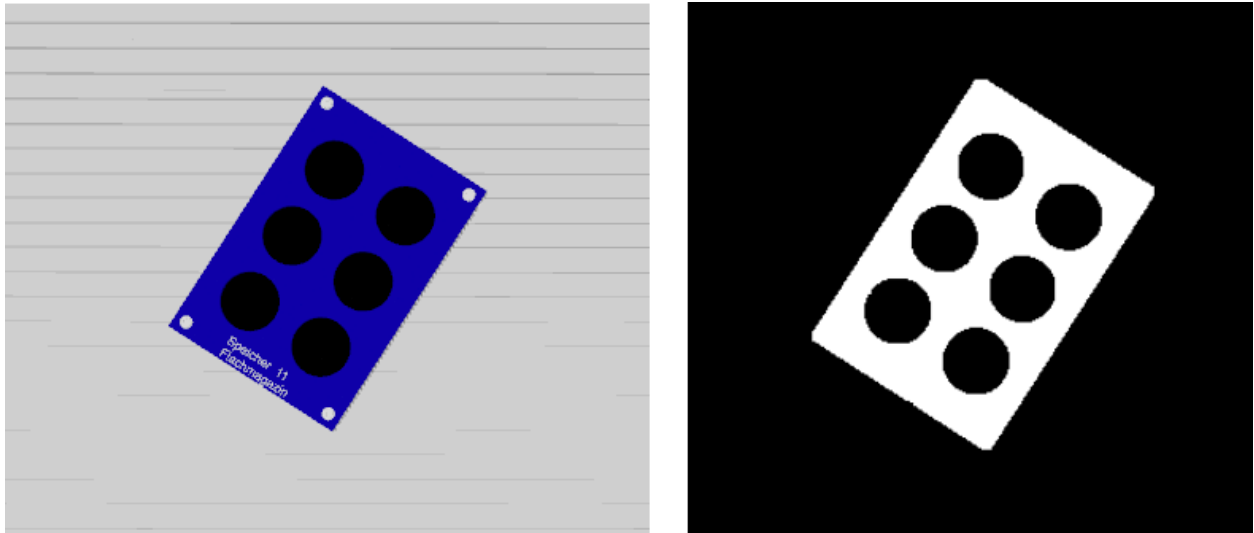


Figure 13: Simulated image without a mask (left hand side) and with mask applied to the image (right hand side). Source: Own image.

Once the blue regions are isolated, the next step is to verify if any of these contrasted objects correspond to the actual storage unit. This is achieved through shape analysis. Given that the storage unit is a rectangular cuboid with known dimensions (220mm x 150mm), its top-down view will appear as a rectangle with an approximate aspect ratio of 22:15. OpenCV provides functions, such as contour detection, which can identify the boundary points of distinct shapes within the masked image. By analyzing the geometric properties of these detected contours (their area, aspect ratio, angles in the corners, and rectangularity)³⁶, the system can determine if a detected blue object matches the expected shape and proportions of the storage unit.

4.2 Storage Localization

Once the storage unit is reliably detected within the image, the next critical step is to determine its precise 3D coordinates in the world reference frame. Several approaches exist for this localization task. One method involves using fiducial markers, such as a QR code with a known position in the world frame and then calculating the storage unit's position relative to the detected marker. However, this approach is dependent on the consistent visibility and integrity of the marker.

Therefore, this project employs an alternative method based on direct image measurements and camera calibration. First, the centroid (middle point) of the detected storage unit in the 2D image (in pixel coordinates) is calculated. The pixel coordinates of the image center are also known, as the image resolution (width and height in pixels) is constant. The 2D displacement of the storage unit's centroid from the image center can then be measured in pixels. To convert these pixel measurements into real-world units (preferable, meters), a scale factor is required. This scale factor can be determined because the physical dimensions of the storage unit are known (its width of 150mm corresponds to a certain number of

³⁶ (doxygen, 2025)

pixels in the image when viewed from a specific distance). By comparing the known physical size to its apparent size in pixels, a pixel-to-meter conversion ratio for calculating distance between the storage's centroid and image center, can be established for objects, using formula below:

$$real_distance = \frac{real_object_width}{image_object_width} * image_distance$$

To transition from these 2D image plane coordinates (now in meters relative to the camera's optical axis) to 3D coordinates in the camera's reference frame, the distance from the camera to the storage unit (the Z-depth) is needed. In this setup, the camera is mounted at a fixed height above the worksurface where the storage unit rests, so this Z-depth is a known, constant value derived from the physical setup and can be confirmed through simulation or documentation.

With the object's (X, Y, Z) coordinates determined relative to the camera's reference frame, and given that the camera's own position and orientation in the world reference frame are fixed and pre-calibrated, a final coordinate transformation (using a homogeneous transformation matrix) is applied to express the storage unit's position in the world frame (see Figure 14 for an illustration of these coordinate systems).

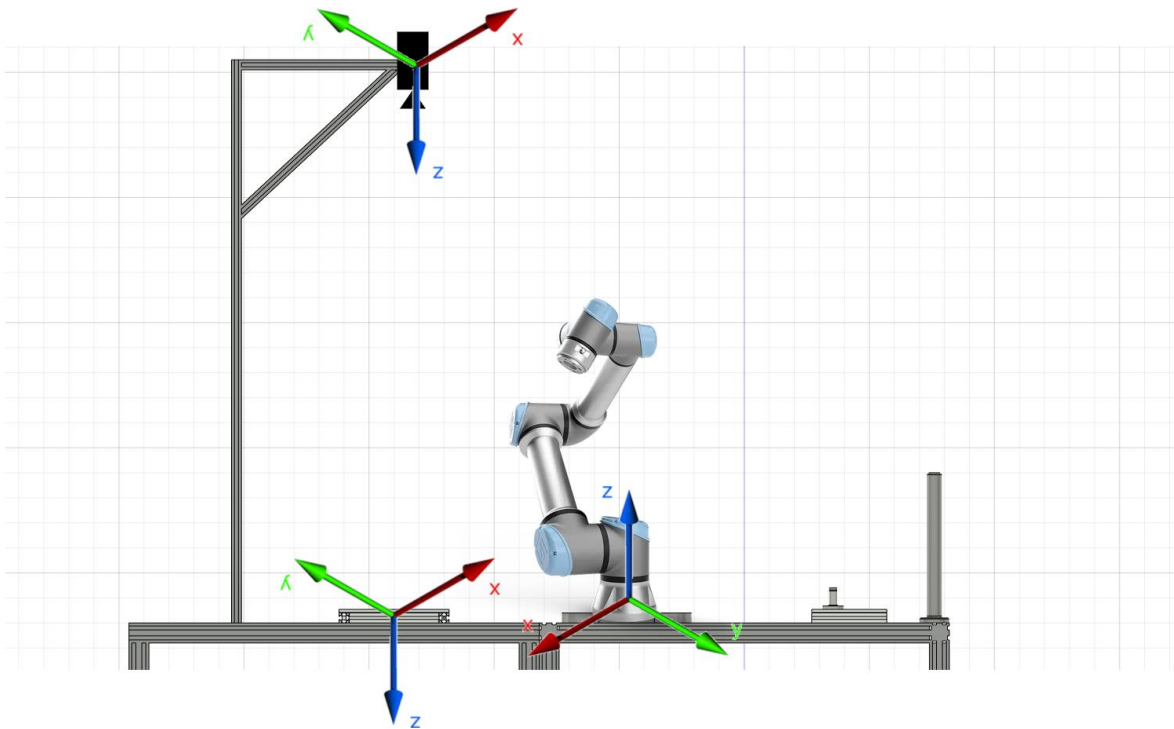


Figure 14: Illustrative image describing the different coordinate systems involved in the setup (world, camera, object). Source: Own image.

The final stage of perception involves identifying the specific locations on the storage unit where workpieces can be placed in this case, the black cylindrical fossettes. To optimize this process and reduce computational load, the image can be cropped to a region of interest (ROI) tightly bounding the detected storage unit. Within this ROI, an algorithm like the Hough Circle Transform is applied to detect circular fossettes. The Hough Circle Transform typically operates by first identifying edges within the image (using

an edge detector) and then searching for circular patterns among these edges ³⁷. This function outputs the 2D pixel coordinates (x, y) of the center of each detected circle and its radius. The detected 2D pixel coordinates of the fossette centers are then transformed into 3D world coordinates using the same depth information and coordinate transformation methods described previously for the storage unit itself. Including one additional step in the beginning, which transforms between 2D circles coordinate systems to 2D storage unit coordination system. The detected radius can be used for verification or dropped, though the actual radius is known from the storage unit's specifications. These 3D fossette locations then serve as potential pick-or-place targets for the robot (see Figure 15).

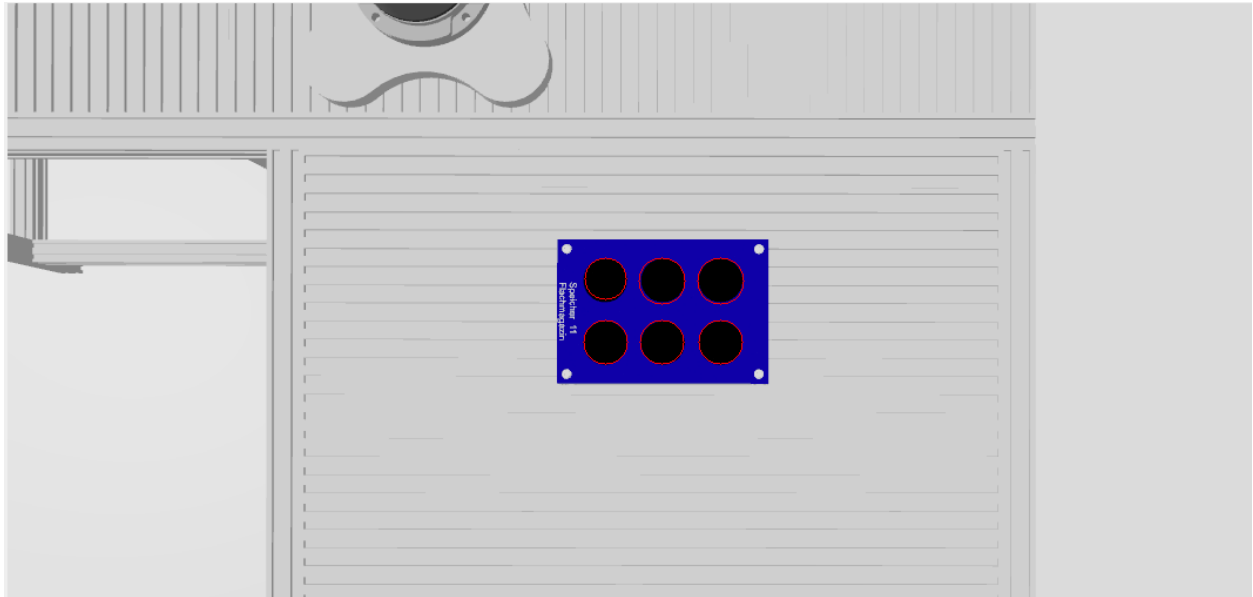


Figure 15: The view from camera simulation, with computed image and detected places for the workpieces, highlighted with thin red circles. Source: Own image.

Described localization approach, when combined with a fixed initial position, clearly defines both the static starting point and the dynamically determined end points for the workpiece's movement.

³⁷ (Hough Circle Transform, 2023)

5 EXAMPLE APPLICATION

This chapter details the integration of the previously described software components and methodologies into an interconnected simulation system. The developed software aims to automatically generate trajectories for manipulating a workpiece from a fixed starting position to a dynamically adapting goal position, the coordinates of which are determined by the real-time perception of its target location.

5.1 Communication Methods in ROS 2

Before diving into the application specifics, it is beneficial to revisit the fundamental node architecture of ROS 2. The ROS graph can be visualized as a network of ROS 2 elements, termed nodes, which concurrently and often independently process data. Each node is typically designed for a single, modular purpose, such as image processing, motion planning, or sensor data computing. Nodes can then exchange data with other nodes through three primary communication mechanisms: topics, services, and actions. This architecture allows complex systems to be decomposed into many smaller, manageable nodes, each responsible for a distinct task ³⁸.

Topics represent one of the most common communication methods in ROS 2. They function like a shared bus over which nodes exchange messages. A node can publish messages to any number of topics and subscribe to messages from any number of topics, potentially creating a complex web of communication between nodes performing singular tasks. For example, the world frame coordinates of a localized storage unit could be published on a topic. The node generating this data might subscribe to an image capture topic (from a live streaming camera), process the video frames to detect and localize the storage unit, and then publish its coordinates. Any other node requiring these coordinates, such as a path planner, can subscribe to this topic to receive the data for further processing. An important feature is that all published data is typically timestamped using a synchronized clock, ensuring temporal consistency across independent nodes (see Figure 16) ³⁹.

³⁸ (Robotics O. , Understanding nodes, 2018)

³⁹ (Robotics O. , Understanding topics, 2018)

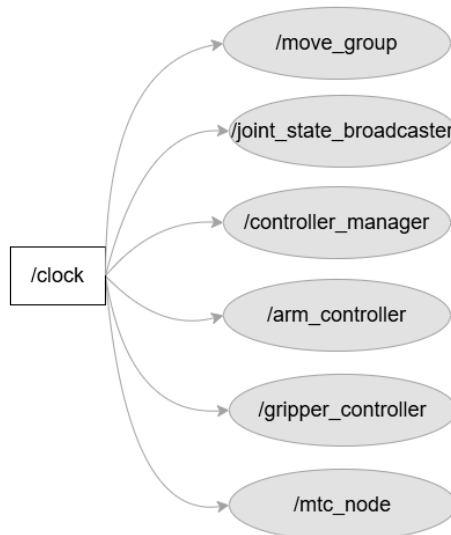


Figure 16: Conceptual illustration of node synchronization using a common clock. Source: Own image.

Services offer another communication method for nodes, but they operate on a call-and-response (request-reply) model, contrasting with the unidirectional publisher-subscriber model of topics. While topics allow nodes to subscribe to continuous data streams based on the publisher's rate, services only provide data when specifically called by a client node. A service client initiates a request (for example, finding an optimal trajectory from this start point to this target point, considering this environment), and a service server (another node) processes this request and returns a single response (in this example, a computed trajectory) ⁴⁰.

Finally, actions are the third communication type in ROS 2 relevant to this project. Actions are designed for long-running tasks and consist of three main parts: a goal, feedback, and a result. They are built upon the underlying topic and service mechanisms. While functionally similar to services in that they involve a client sending a request (a goal) to a server, actions differ significantly in that they can be canceled at any time during execution. Furthermore, actions provide a continuous stream of feedback about the task's progress, whereas services typically return only a single, final response. An action client sends a goal to an action server; the server acknowledges the goal and then provides periodic feedback and a final result. Actions are particularly well-suited for tasks like trajectory execution. In this scenario, a known trajectory (perhaps obtained as a response from a planning service) is divided into a sequence of waypoints, which are then sent as goals to an action server controlling the robot. The server executes the motion to each waypoint, providing steady feedback on the robot's joint states, velocities, accelerations, or measured torques ⁴¹.

These communication tools, such as topics, services, and actions, are the primary methods for internode communication in ROS 2. This structured approach enables the construction of highly complex software

⁴⁰ (Robotics O. , Understanding services, 2018)

⁴¹ (Robotics O. , Understanding actions, 2018)

systems from smaller, modular "bricks" (nodes), which is a key reason ROS 2 was selected as the middleware for this project. A well-orchestrated system of nodes, each performing its specific function and communicating effectively, forms the backbone of the application.

5.2 Communication Architecture and Operational Flow

This subchapter consolidates the knowledge presented throughout this thesis, describing the operational flow and internal processes of the developed software system as it executes its primary task. The task, as previously established, involves motion planning for a robot to grasp a well-defined workpiece from a fixed storage location within its workspace, transporting it to a target point which position is dynamically localized, and then place the workpiece. It is important to point out that, with the exception of the initial fixed storage location, no intermediate path points are manually defined by a human. Instead, the software autonomously calculates how to approach and pick the workpiece, how to lift and move it without colliding with other objects or the robot itself, and how to place it safely. The only predefined information includes the known parameters and dimensions of the workspace, the workpiece, and the storage units.

The foundational step is to create an accurate digital representation of the real world within the simulation environment. This is managed, in part, by the *planning_scene* node (often part of or closely interacting with *move_group*), which publishes information about the environment's geometry and collision objects via ROS 2 topics.

The digital twin of the UR5 robot and its workspace, including all its kinematic parameters, joint limits, and collision geometry, is defined within the simulation and configuration files. This robot description data is loaded by and managed within the *move_group* node. The *move_group* node is a core component of the MoveIt 2 motion planning framework. It provides a high-level interface for controlling and planning the motion of the robot's manipulator by allowing users or other nodes to set joint space or Cartesian pose goals, request motion plans, and command their execution.

The real-time state of the physical or simulated robot, including joint positions, velocities, accelerations, and effort (torque) is monitored and broadcast by the *joint_state_broadcaster* (a part of *ros2_control* components). This node continuously reads sensor data from the robot hardware (or simulated equivalent) and publishes it as joint state messages, to which other nodes like a controller manager subscribes.

For executing complex, multi-stage tasks, the *move_group* node can interface with the *mtc_node* (MoveIt Task Constructor node) often via ROS 2 actions. The *mtc_node* would then typically communicate with the *controller_manager* (another *ros2_control* component) using actions to send these individual subtasks for execution and monitor their feedback.

For communication ROS 2 software with the UR 5 (physical and simulation), specific driver components are necessary. These are managed within the controller manager, which loads and activates controllers like *arm_controller* – for robot execution, and *gripper_controller* – to control the end effector, open/close.

Finally, the *opencv* node (a custom node developed for this project) bridges the real-world visual information with the digital simulation and planning system. This node encapsulates the image processing

functionalities described in a previous chapter. It continuously processes camera input to detect and localize the target storage unit, publishing its coordinates (relative to the world coordinate system) via a ROS 2 topic. This allows any other interested node, such as the path planner or task constructor, to subscribe and receive real-time updates on the target's position. Figure 17 provides a simplified diagram of this communication architecture.

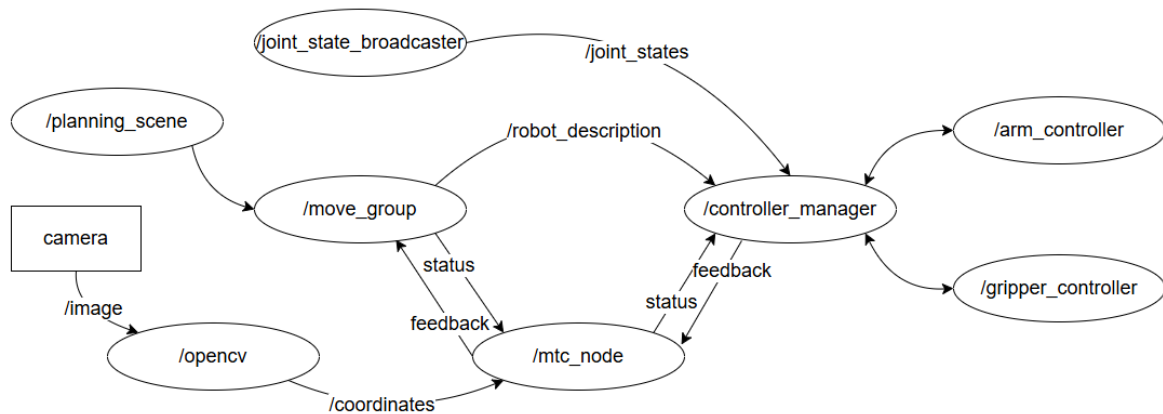


Figure 17: Simplified communication architecture illustrating the ROS 2 nodes and their communication methods within the developed system. Source: Own image.

This node graph represents a simplified overview of how the custom software processes data. A significant advantage of the ROS 2 architecture is its inherent support for parallel processing through multithreading and multi-process execution, allowing each node to operate largely independently. For instance, when a path planning request is made, the MoveIt task constructor gets target position from opencv node and generates multiple solution trajectories (the number of solutions can be configured by the user based on desired computation time and outcome quality). Each of these trajectories can then be evaluated against a cost function, as discussed previously, enabling the system to determine and select the most suitable solution from the available options. After finding the best option among the solutions, the task is executed.

5.3 Outcome Software

The integrated node infrastructure described concludes in a functional software system. This system can dynamically localize the target storage unit within the Gazebo simulated environment (which serves as a representation of the real world) and store its determined position. Based on the overall task requirements, the MoveIt Task Constructor (MTC) is configured, potentially utilizing structures like serial or parallel containers, to break down the complex manipulation task (pick and place) into a sequence of manageable subtasks.

When a planning request is initiated, MTC, in conjunction with the underlying OMPL planners, searches for solutions. If parallel containers or multiple solution strategies are employed, MTC can explore different approaches concurrently and may generate several candidate solutions or solution fragments for various stages of the task. These are then evaluated, and often, multiple complete, valid trajectories might be found. For instance, Figure 18 (right-hand panel) conceptually illustrates how a list of potential solutions might be

presented, with successful attempts marked (with a green tick) and failed attempts indicated (with a red cross).

From the pool of successful candidate trajectories, the system can then select a preferred one based on a cost analysis. Typically, MTC allows for cost evaluation at various levels, and the solutions with the lowest accumulated cost are considered more optimal according to the defined cost functions. In this implementation, the software is designed to identify a set of, for example, five distinct valid paths, ranked by their associated costs. Figure 18 presents the RViz visualization where the "cheapest" trajectory (optimal in that particular planning iteration based on cost) are displayed. The various arrows and colored segments in the visualization represent the different stages or sub-trajectories planned by MTC, and the sequence of "hands" or end-effector poses illustrates the key intermediate points along this selected optimal trajectory that the robot will execute.

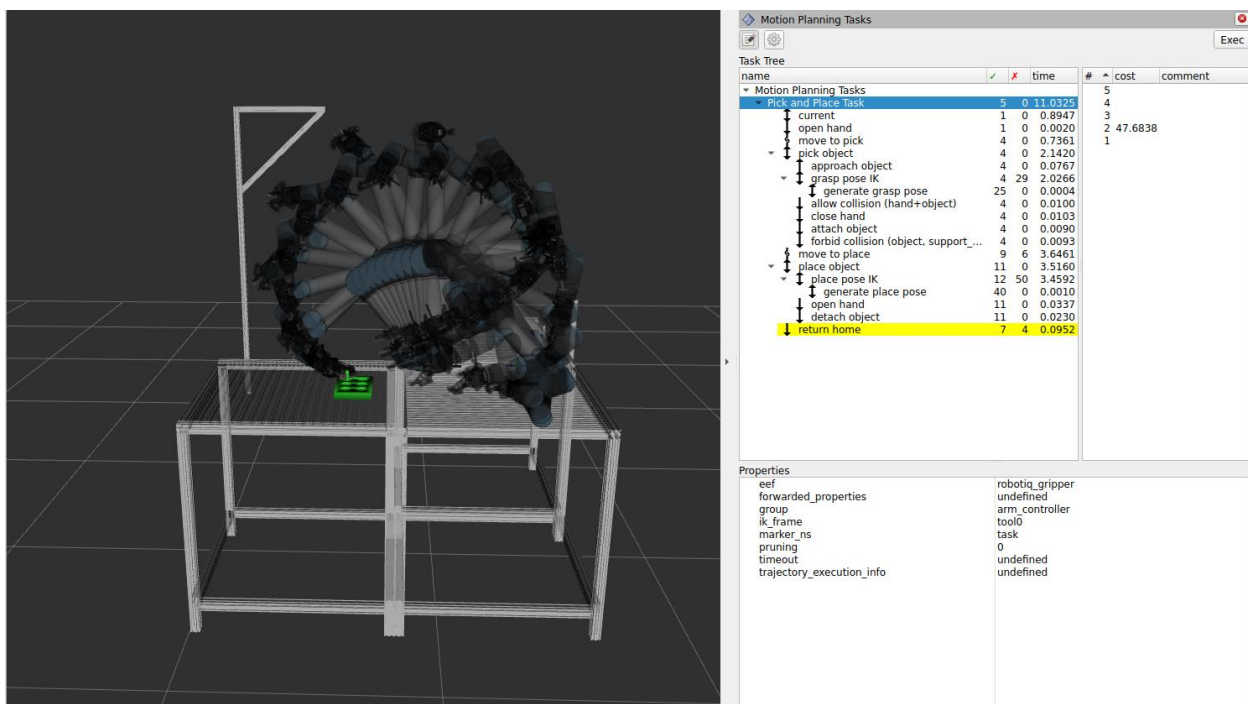


Figure 18: RViz visualization showing one of the cheapest (most optimal) trajectories found by the MoveIt Task Constructor (MTC) for a pick and place task. Source: Own image.

6 CONCLUSIONS AND FUTURE WORK

ROS 2 as a software framework and MoveIt as a path planning plugin are very powerful tools, with a potential for future implementation. The proof of concept acknowledges its possible usability in the field of industrial robotics. Despite the short time and high entrance level to understand the straightforward architecture of ROS 2, it shows its nature in fast prototyping, code reuse or instinctive communication models for different tasks. How it was shown in the beginning, ROS 2 gives a possibility to bridge digital with real world.

Therefore, the findings presented in this bachelor should be interpreted as indicative of the considerable potential for further exploration and development in this domain. While this work primarily validates the system within a controlled simulated environment, future efforts could focus on addressing the additional complexities and limitations that may arise when transitioning to and operating within real-world industrial settings, by extending the research beyond simulation to practical field deployment.

BIBLIOGRAPHY

Planners Available in MoveIt. (kein Datum). Von MoveIt: <https://moveit.ai/documentation/planners/> abgerufen

(29. 03 2025). Von MRobotics: <https://robotics.umich.edu/research/focus-areas/motion-planning/> abgerufen

A/S, U. R. (2024). *Universal Robots ROS 2 Driver Documentation.* Von Welcome to the Universal Robots ROS 2 driver documentation: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/ abgerufen

About offline programming. (kein Datum). Von Robotmaster offline programming software for robotics : <https://www.robotmaster.com/about-offline-programming/> abgerufen

Al-Batati, A. S., Koubaa, A., & Abdelkader, M. (2024). *ROS 2 Key Challenges and Advances: A Survey of ROS 2 Research, Libraries, and Applications.* Rijad: Preprints.org.

automaticaddison. (29. 08 2024). *Getting Started with the MoveIt 2 Task Constructor.* Von Automatic Addison: <https://automaticaddison.com/getting-started-with-the-moveit-2-task-constructor/> abgerufen

Dellaert, F., & Hutchinson, S. (2023). 7.5. *Trajectory Optimization.* Von Robotics {book}: https://www.roboticsbook.org/S75_drone_planning.html abgerufen

doxygen. (14. 05 2025). *Contours Begin.* Von OpenCV Open Source Computer Vision docs: https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html abgerufen

fmauch. (22. 03 2021). *Universal_Robots_ExternalControl_URCap.* Von github: https://github.com/UniversalRobots/Universal_Robots_ExternalControl_URCap abgerufen

Gîrbacia, F., Duguleană, M., & Stavăr, A. (2011). *Off-line programming of industrial robots using co-located environments.* Sinaia: Transilvania University of Brasov.

Henry, N. (17. 05 2025). *Understanding Industrial Robot Motion Types.* Von SolisPLC: <https://www.solisplc.com/tutorials/industrial-robot-motion-types#:~:text=Linear%20motions%20move%20the%20robot%20in%20a,on%20the%20robot%20application%20and%20application%20environment> abgerufen

Hershberger, D., Gossow, D., & Faust, J. (27. 04 2019). *rviz Documentation.* Von docs.ros.org: <https://docs.ros.org/en/lunar/api/rviz/html/c++/index.html> abgerufen

Hough Circle Transform. (2023). Von TheAILearner: <https://theailearner.com/tag/hough-gradient-method/> abgerufen

HSL and HSV. (25. 03 2025). Von Wikipedia, The Free Encyclopedia: https://en.wikipedia.org/wiki/HSL_and_HSV abgerufen

- Kuffner, J. J., & LaValle, S. M. (2000). *RRT-Connect: An Efficient Approach to Single-Query Path Planning*. San Francisco: Institute of Electrical and Electronics Engineers.
- Lab-Kavraki. (2025). *Open Motion Planning Library: A Primer*. Houston: Rice University.
- Marcelina. (01. 07 2022). *Motion Planning vs Path Planning*. Von Medium: <https://medium.com/shade-robotics/motion-planning-vs-path-planning-e29ced760b90> abgerufen
- OMPL. (2022). *The Open Motion Planning Library*. Von OMPL kavrakilab: <https://ompl.kavrakilab.org/> abgerufen
- OpenCV-team. (2000). *OpenCV*. Von OpenCV.org: <https://opencv.org/> abgerufen
- Owen-Hill, A. (14. 09 2015). *Kinematics: Why Robots Move Like They Do*. Von ROBOTIQ: <https://blog.robotiq.com/kinematics-why-robots-move-like-they-do> abgerufen
- Robotics, O. (2002). *gazebo home*. Von gazebo: <https://gazebo.org/home> abgerufen
- Robotics, O. (10. 09 2018). *Understanding actions*. Von ROS 2 Documentation: Rolling: <https://docs.ros.org/en/rolling/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html> abgerufen
- Robotics, O. (10. 09 2018). *Understanding nodes*. Von ROS 2 Documentation: Rolling: <https://docs.ros.org/en/rolling/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html> abgerufen
- Robotics, O. (10. 09 2018). *Understanding services*. Von ROS 2 Documentation: Rolling: <https://docs.ros.org/en/rolling/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html> abgerufen
- Robotics, O. (10. 09 2018). *Understanding topics*. Von ROS 2 Documentation: Rolling: <https://docs.ros.org/en/rolling/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html> abgerufen
- Robotics, O. (2021). *ROS - Robot Operating System*. Von ROS.org: <https://www.ros.org/> abgerufen
- Robotics, O. (2025). *ROS 2 Documentation: Jazzy*. Von docs.ros.org: <https://docs.ros.org/en/jazzy/index.html> abgerufen
- Robotics, P. (15. 01 2024). *MoveIt Task Constructor*. Von MoveIt PickNik AI: https://moveit.picknik.ai/main/doc/concepts/moveit_task_constructor/moveit_task_constructor.html abgerufen
- Robotics, P. (2025). *Motion Planning*. Von MoveIt: https://moveit.picknik.ai/main/doc/concepts/motion_planning.html abgerufen
- Robotics, P. (2025). *MoveIt 2 Documentation*. Von MoveIt: <https://moveit.picknik.ai/main/index.html> abgerufen

Robotics, P. (2025). *Time Parameterization*. Von Movelt Humble:

https://moveit.picknik.ai/humble/doc/examples/time_parameterization/time_parameterization_tutorial.html abgerufen

Said, A., Talj, R., Francis, C., & Shraim, H. (2022). *Tuning and Costs Analysis for a Trajectory Planning*. France: Hal science.

Shaw, H. (22. 02 2024). *Whats the difference between industry 4.0 and 3.0*. Von ROWSE:

<https://www.rowse.co.uk/blog/post/whats-the-difference-between-industry-4.0-and-industry-3.0> abgerufen

Sphinx. (2017). *Costs and Constraints*. Von Industrial training master: https://industrial-training-master.readthedocs.io/en/melodic/_source/demo3/Costs-and-Constraints.html abgerufen

Veeramachaneni, S. (21. 05 2024). *Exploring Color Manipulation in Images with OpenCV: Masking vs. Custom Color Selection*. Von Medium: <https://medium.com/@sunil.veera/exploring-color-manipulation-in-images-with-opencv-masking-vs-custom-color-selection-803d9742969c> abgerufen

What is offline robot programming (OLP)? A complete guide. (28. 04 2025). Von Standard Bots:

<https://standardbots.com/blog/offline-robot-programming> abgerufen

What is robot programming. (01. 02 2022). Von ABAGY: <https://abagy.com/what-is-robot-programming> abgerufen